

CSA0603-DESIGN AND ANALYSIS OF ALGORITHM(CHAPTER 1- PRACTICALS)

Exp1: First Palindromic String in an Array

```
q1.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q1.py (3.13.5)
File Edit Format Run Options Window Help

# Function to check palindrome
def is_palindrome(word):
    return word == word[::-1]

# Input
words = input("Enter words separated by space: ").split()

# Find first palindrome
result = ""
for word in words:
    if is_palindrome(word):
        result = word
        break

# Output
print("First Palindromic String:", result)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q1.py =====
Enter words separated by space: abc car ada racecar cool
First Palindromic String: ada
>>>
```

Exp 2 : Count Elements Common Between Two Arrays

```
q2.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q2.py (3.13.5)
File Edit Format Run Options Window Help

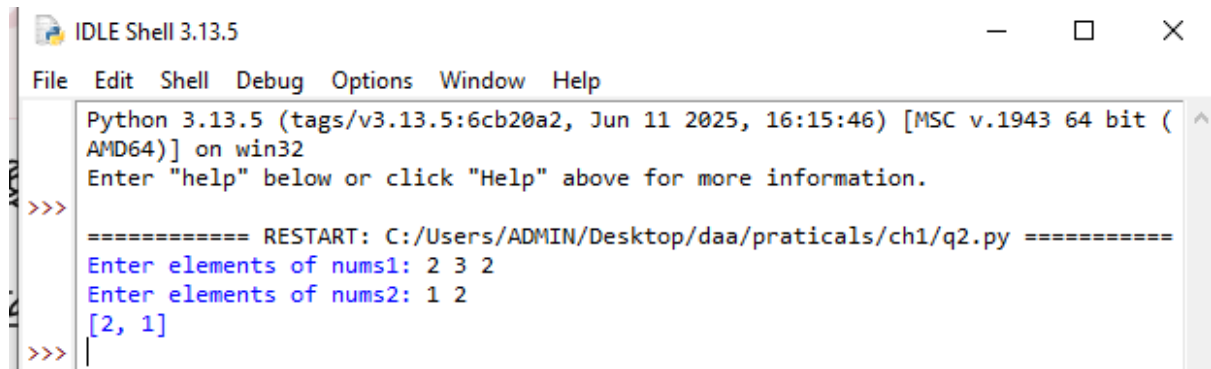
# Input
nums1 = list(map(int, input("Enter elements of nums1: ").split()))
nums2 = list(map(int, input("Enter elements of nums2: ").split()))

answer1 = 0
answer2 = 0

# Count elements of nums1 present in nums2
for num in nums1:
    if num in nums2:
        answer1 += 1

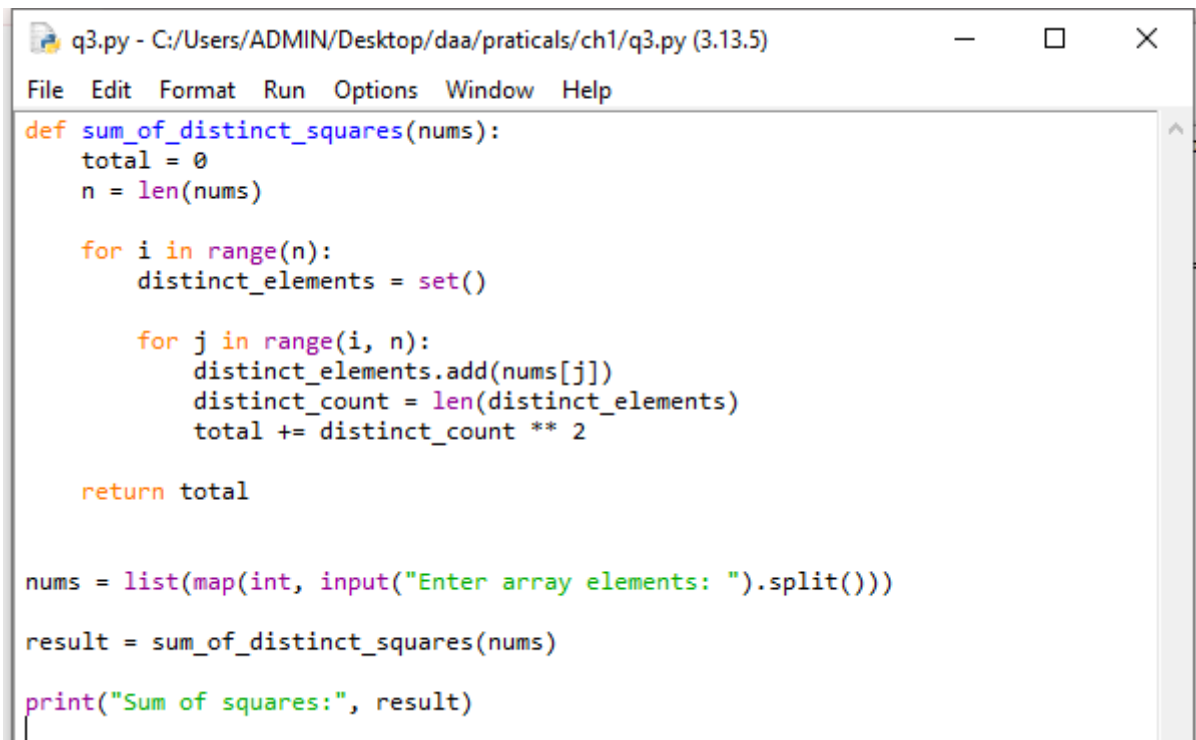
# Count elements of nums2 present in nums1
for num in nums2:
    if num in nums1:
        answer2 += 1

# Output
print([answer1, answer2])
```



```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q2.py =====
Enter elements of nums1: 2 3 2
Enter elements of nums2: 1 2
[2, 1]
>>>
```

Exp 3: Sum of Squares of Distinct Counts of All Subarrays



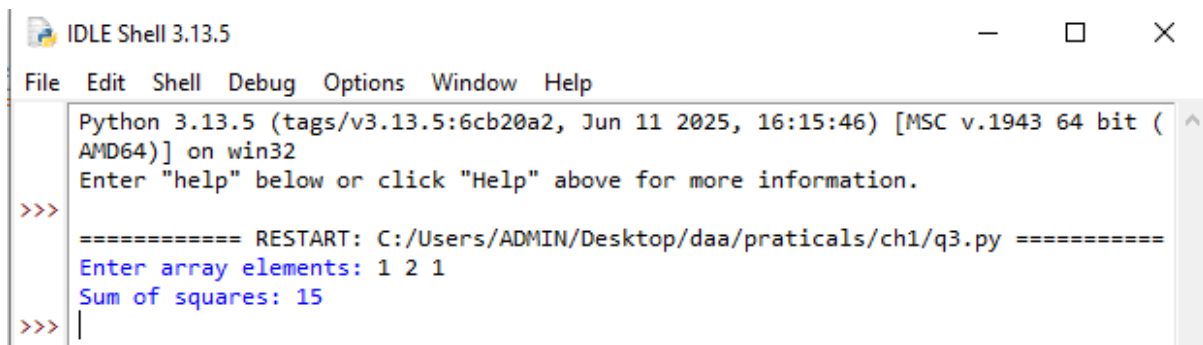
```
q3.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q3.py (3.13.5)
File Edit Format Run Options Window Help
def sum_of_distinct_squares(nums):
    total = 0
    n = len(nums)

    for i in range(n):
        distinct_elements = set()

        for j in range(i, n):
            distinct_elements.add(nums[j])
            distinct_count = len(distinct_elements)
            total += distinct_count ** 2

    return total

nums = list(map(int, input("Enter array elements: ").split()))
result = sum_of_distinct_squares(nums)
print("Sum of squares:", result)
```



```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q3.py =====
Enter array elements: 1 2 1
Sum of squares: 15
>>>
```

Exp 4: Count Equal Pairs with Index Product Divisible by K

```
q4.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q4.py (3.13.5)
File Edit Format Run Options Window Help

def count_pairs(nums, k):
    count = 0
    n = len(nums)

    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count

nums = list(map(int, input("Enter array elements: ").split()))
k = int(input("Enter k value: "))

result = count_pairs(nums, k)

print("Number of valid pairs:", result)

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q4.py =====
Enter array elements: 3 1 2 2 2 1 3
Enter k value: 2
Number of valid pairs: 4
```

Exp 5: Find the Maximum Element in an Array

```
q5.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q5.py (3.13.5)
File Edit Format Run Options Window Help

def find_maximum(arr):
    maximum = arr[0]

    for value in arr[1:]:
        if value > maximum:
            maximum = value

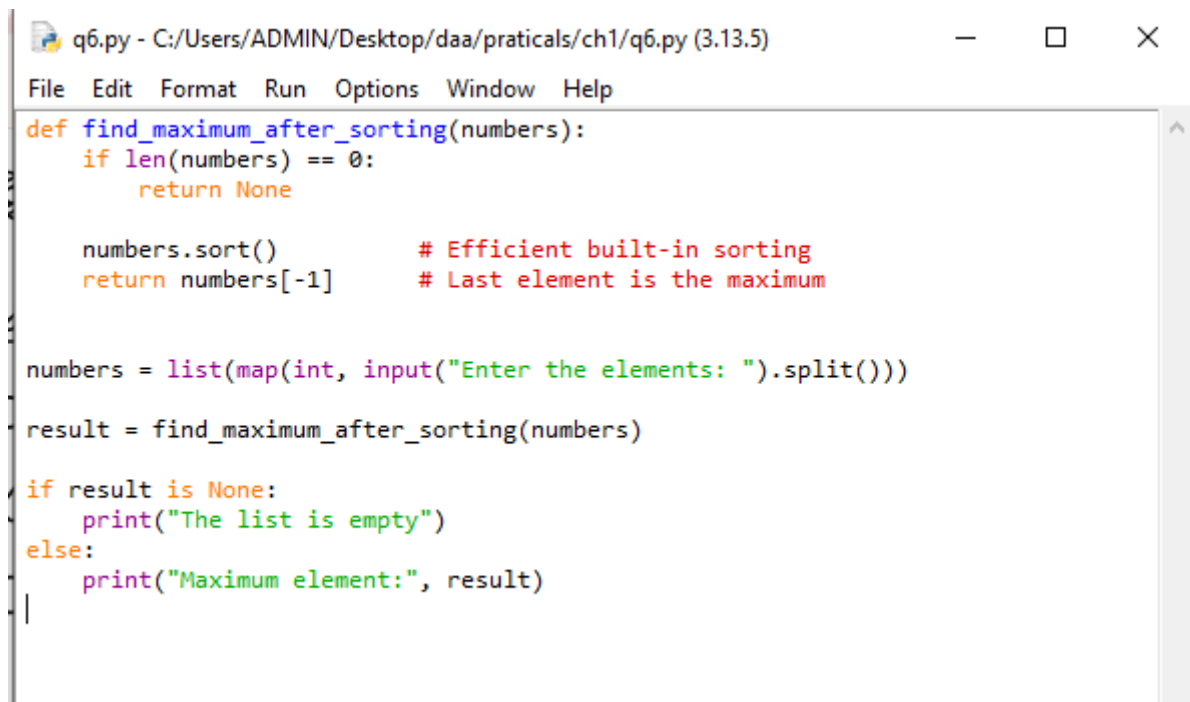
    return maximum

arr = list(map(int, input("Enter array elements: ").split()))

if len(arr) == 0:
    print("Array cannot be empty")
else:
    result = find_maximum(arr)
    print("Maximum element:", result)
```

```
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q5.py =====  
Enter array elements: 1 2 3 4 5  
Maximum element: 5  
|
```

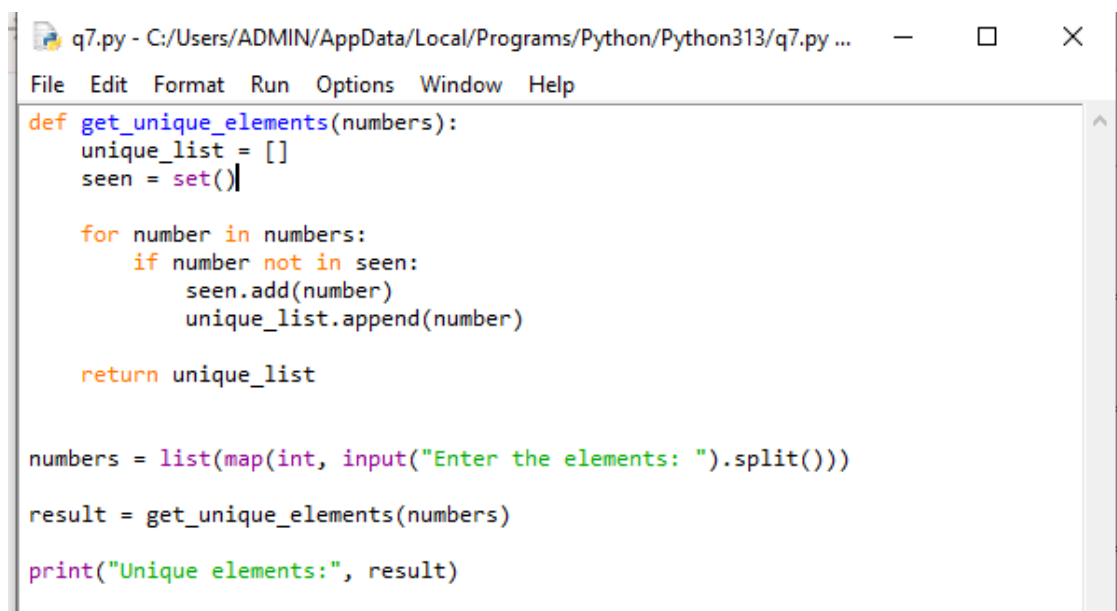
Exp 6: Sort the List and Find the Maximum Element



```
q6.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q6.py (3.13.5)  
File Edit Format Run Options Window Help  
def find_maximum_after_sorting(numbers):  
    if len(numbers) == 0:  
        return None  
  
    numbers.sort()          # Efficient built-in sorting  
    return numbers[-1]      # Last element is the maximum  
  
numbers = list(map(int, input("Enter the elements: ").split()))  
result = find_maximum_after_sorting(numbers)  
  
if result is None:  
    print("The list is empty")  
else:  
    print("Maximum element:", result)  
|
```

```
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q6.py =====  
Enter the elements: 2 3 4 6 7  
Maximum element: 7  
|
```

Exp 7: Find Unique Elements in a List



```
q7.py - C:/Users/ADMIN/AppData/Local/Programs/Python/Python313/q7.py ...  
File Edit Format Run Options Window Help  
def get_unique_elements(numbers):  
    unique_list = []  
    seen = set()  
  
    for number in numbers:  
        if number not in seen:  
            seen.add(number)  
            unique_list.append(number)  
  
    return unique_list  
  
numbers = list(map(int, input("Enter the elements: ").split()))  
result = get_unique_elements(numbers)  
print("Unique elements:", result)  
|
```

```
==== RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python313/q7.py ====
Enter the elements: 3 7 3 5 2 5 9 2
Unique elements: [3, 7, 5, 2, 9]
|
```

Exp 8: Sort an Array Using Bubble Sort

```
q8.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q8.py (3.13.5)
File Edit Format Run Options Window Help
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        swapped = False

        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

arr = list(map(int, input("Enter the array elements: ").split()))
sorted_array = bubble_sort(arr)
print("Sorted array:", sorted_array)

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q8.py =====
Enter the array elements: 64 34 25 12 22 11 90
Sorted array: [11, 12, 22, 25, 34, 64, 90]
|
```

Exp 9: Search an Element Using Binary Search

```
q9.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q9.py (3.13.5)
File Edit Format Run Options Window Help
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1

arr = list(map(int, input("Enter the array elements: ").split()))
key = int(input("Enter the key element: "))

arr.sort()

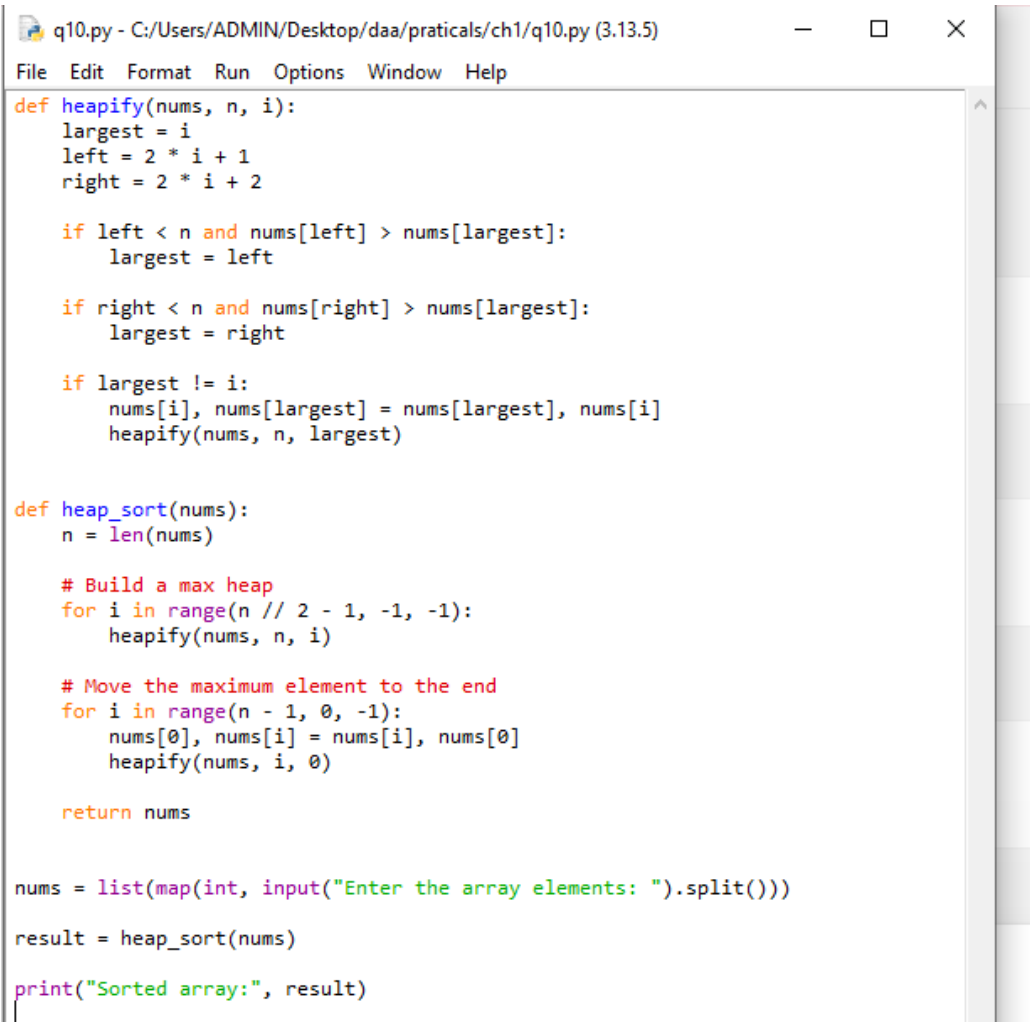
print("Sorted array:", arr)

position = binary_search(arr, key)

if position != -1:
    print("Element", key, "is found at position", position + 1)
else:
    print("Element", key, "is not found")
|
```

```
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q9.py =====  
Enter the array elements: 3 4 6 -9 10 8 9 30  
Enter the key element: 10  
Sorted array: [-9, 3, 4, 6, 8, 9, 10, 30]  
Element 10 is found at position 7
```

Exp 10: Sort an Array in $O(n \log n)$ Time Using Heap Sort



```
q10.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q10.py (3.13.5)  
File Edit Format Run Options Window Help  
def heapify(nums, n, i):  
    largest = i  
    left = 2 * i + 1  
    right = 2 * i + 2  
  
    if left < n and nums[left] > nums[largest]:  
        largest = left  
  
    if right < n and nums[right] > nums[largest]:  
        largest = right  
  
    if largest != i:  
        nums[i], nums[largest] = nums[largest], nums[i]  
        heapify(nums, n, largest)  
  
def heap_sort(nums):  
    n = len(nums)  
  
    # Build a max heap  
    for i in range(n // 2 - 1, -1, -1):  
        heapify(nums, n, i)  
  
    # Move the maximum element to the end  
    for i in range(n - 1, 0, -1):  
        nums[0], nums[i] = nums[i], nums[0]  
        heapify(nums, i, 0)  
  
    return nums  
  
nums = list(map(int, input("Enter the array elements: ").split()))  
result = heap_sort(nums)  
print("Sorted array:", result)
```

```
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q10.py =====  
Enter the array elements: 5 2 3 1  
Sorted array: [1, 2, 3, 5]
```

Exp 11: Count Paths to Move a Ball Out of a Grid

```
q11.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q11.py (3.13.5)
File Edit Format Run Options Window Help
def find_paths(m, n, max_moves, start_row, start_col):
    dp = [[0] * n for _ in range(m)]
    dp[start_row][start_col] = 1
    paths = 0

    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for _ in range(max_moves):
        next_dp = [[0] * n for _ in range(m)]

        for row in range(m):
            for col in range(n):
                if dp[row][col] == 0:
                    continue

                for dr, dc in directions:
                    new_row = row + dr
                    new_col = col + dc

                    if new_row < 0 or new_row >= m or new_col < 0 or new_col >= n:
                        paths += dp[row][col]
                    else:
                        next_dp[new_row][new_col] += dp[row][col]

        dp = next_dp

    return paths

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))
N = int(input("Enter maximum moves: "))
i = int(input("Enter starting row: "))
j = int(input("Enter starting column: "))

print("Number of paths:", find_paths(m, n, N, i, j))
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q11.py =====
Enter number of rows: 2
Enter number of columns: 2
Enter maximum moves: 2
Enter starting row: 0
Enter starting column: 0
Number of paths: 6
>>>
```

Exp 12: House Robber in a Circular Arrangement

```
q12.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q12.py (3.13.5)
File Edit Format Run Options Window Help

def rob_linear(houses):
    previous_two = 0
    previous_one = 0

    for money in houses:
        current = max(previous_one, previous_two + money)
        previous_two = previous_one
        previous_one = current

    return previous_one

def rob_circular(nums):
    n = len(nums)

    if n == 0:
        return 0

    if n == 1:
        return nums[0]

    exclude_last = rob_linear(nums[:-1])
    exclude_first = rob_linear(nums[1:])

    return max(exclude_last, exclude_first)

nums = list(map(int, input("Enter money in each house: ").split()))
result = rob_circular(nums)

print("Maximum money that can be robbed:", result)
```

```
===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q12.py =====
Enter money in each house: 2 3 2
Maximum money that can be robbed: 3
```

Exp 13: Count Distinct Ways to Climb Stairs

```
q13.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q13.py (3.13.5)
File Edit Format Run Options Window Help

def climb_stairs(n):
    if n <= 1:
        return 1

    first = 1
    second = 2

    for _ in range(3, n + 1):
        current = first + second
        first = second
        second = current

    return second

n = int(input("Enter the number of steps: "))
print("Number of distinct ways:", climb_stairs(n))
```


Exp 14: Count Unique Paths in a Grid

```
q14.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q14.py (3.13.5)
File Edit Format Run Options Window Help

def unique_paths(m, n):
    dp = [1] * n

    for row in range(1, m):
        for col in range(1, n):
            dp[col] = dp[col] + dp[col - 1]

    return dp[n - 1]

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

print("Number of unique paths:", unique_paths(m, n))

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q14.py =====
Enter number of rows: 7
Enter number of columns: 3
Number of unique paths: 28
```

Exp 15: Find Large Groups in a String

```
q15.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q15.py (3.13.5)
File Edit Format Run Options Window Help

def large_group_positions(s):
    result = []
    start = 0

    for end in range(len(s)):
        if end == len(s) - 1 or s[end] != s[end + 1]:
            group_length = end - start + 1

            if group_length >= 3:
                result.append([start, end])

            start = end + 1

    return result

s = input("Enter the string: ")

print("Large group intervals:", large_group_positions(s))
```

```

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q15.py =====
Enter the string: abxxxxzzy
Large group intervals: [[3, 6]]
|

```

Exp 16: Conway's Game of Life

```

q16.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q16.py (3.13.5)
File Edit Format Run Options Window Help

def game_of_life(board):
    rows = len(board)
    cols = len(board[0])

    directions = [
        (-1, -1), (-1, 0), (-1, 1),
        (0, -1),      (0, 1),
        (1, -1),  (1, 0),  (1, 1)
    ]

    next_board = [[0] * cols for _ in range(rows)]

    for row in range(rows):
        for col in range(cols):
            live_neighbors = 0

            for dr, dc in directions:
                new_row = row + dr
                new_col = col + dc

                if 0 <= new_row < rows and 0 <= new_col < cols:
                    live_neighbors += board[new_row][new_col]

            if board[row][col] == 1:
                if live_neighbors == 2 or live_neighbors == 3:
                    next_board[row][col] = 1
            else:
                if live_neighbors == 3:
                    next_board[row][col] = 1

    return next_board

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

board = []

print("Enter the board rows:")

```

```

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q16.py =====
Enter number of rows: 4
Enter number of columns: 3
Enter the board rows:
0 1 0
0 0 1
1 1 1
0 0 0
Next state:
[0, 0, 0]
[1, 0, 1]
[0, 1, 1]
[0, 1, 0]
|

```

Exp 17: Champagne Tower

```
q17.py - C:/Users/ADMIN/Desktop/daa/practicals/ch1/q17.py (3.13.5)
File Edit Format Run Options Window Help
def champagne_tower(poured, query_row, query_glass):
    tower = [[0.0] * (query_row + 2) for _ in range(query_row + 2)]

    tower[0][0] = poured

    for row in range(query_row):
        for glass in range(row + 1):
            overflow = max(0.0, tower[row][glass] - 1.0)

            tower[row + 1][glass] += overflow / 2
            tower[row + 1][glass + 1] += overflow / 2

    return min(1.0, tower[query_row][query_glass])

poured = int(input("Enter number of cups poured: "))
query_row = int(input("Enter query row: "))
query_glass = int(input("Enter query glass: "))

result = champagne_tower(poured, query_row, query_glass)

print("Glass fullness:", format(result, ".5f"))

===== RESTART: C:/Users/ADMIN/Desktop/daa/practicals/ch1/q17.py =====
Enter number of cups poured: 1
Enter query row: 1
Enter query glass: 1
Glass fullness: 0.00000
```